PROVIDER REQUIREMENTS

BitScan BitRPC Method Requirements

A professional summary of the Bitcoin RPC methods BitScan already uses, the application features they power, and the indexed capabilities needed to remove Tatum from the architecture.

Goal: make BitRPC the only Bitcoin data provider for BitScan.

Current stack: Next.js frontend -> Rust API -> BitRPC JSON-RPC.

Remaining gap: historical transaction lookup without block context and address-level indexing.

Current BitRPC Usage

BitScan connects to BitRPC as a Bitcoin Core JSON-RPC provider. The Rust backend calls BitRPC from `explorer/src/rpc.rs` through `state.rpc.call(method, params)`. The frontend does not call BitRPC directly.

```
Next.js frontend -> Rust API -> BitRPC JSON-RPC -> Bitcoin node/provider
```

6

BitRPC call patterns currently used by the backend

5

Core Bitcoin RPC methods powering blocks, mempool, and tx pages

3

Indexer capabilities still supplied by Tatum

Important: BitRPC already supports the core explorer flow well. The remaining dependency on Tatum is not for normal block data; it is for indexed transaction and address data.

BitRPC Methods Currently Called

Method	Params Used	Used For	Rust Location
`getblockchaininfo`	`[]`	Chain tip height, best block hash, chain name, verification progress, health checks, and confirmation calculations.	`blocks.rs` `health.rs` `transactions.rs`
`getmempoolinfo`	`[]`	Mempool transaction count, mempool bytes, memory usage, and minimum fee rate.	`blocks.rs`
`getblockhash`	`[height]`	Converts a block height into a block hash for search, block pages, and latest-block pagination.	`blocks.rs`
`getblock`	`[hash, 1]`	Fetches block details: height, hash, previous/next hash, time, merkle root, nonce, bits, difficulty, size, weight, confirmations, and txids.	`blocks.rs` `search.rs`
`getrawtransaction`	`[txid, true, block_hash]`	Fetches decoded transactions from a known block for block transaction summaries and transaction details.	`blocks.rs` `transactions.rs`
`getrawtransaction`	`[txid, true]`	Attempts direct transaction lookup and previous-output lookup. This requires txindex or equivalent transaction indexing.	`transactions.rs` `search.rs`

What Each Method Powers

`getblockchaininfo`

Used by `GET /api/tip`, `GET /api/blocks`, `GET /health`, and transaction confirmation fallback logic.

Required fields: `blocks`, `bestblockhash`, `chain`, `verificationprogress`.

`getmempoolinfo`

Used by `GET /api/mempool` for the mempool card and minimum fee display.

Required fields: `size`, `bytes`, `usage`, `mempoolminfee`.

`getblockhash`

Used for block height search, block detail lookup by height, and latest block listing.

```
[969995]
```

`getblock`

Used by block pages, latest block listing, block transaction pagination, and search classification for 64-character hashes.

Required fields: `height`, `hash`, `previousblockhash`, `nextblockhash`, `time`, `confirmations`, `merkleroot`, `nonce`, `bits`, `difficulty`, `size`, `weight`, `tx`.

`getrawtransaction`

Used for transaction details, inputs, outputs, total output value, and fee calculations.

```
["txid", true, "block_hash"]
["txid", true]
```

Why Tatum Is Still Used

The current BitRPC methods are enough for chain tip, mempool status, latest blocks, block details, block transaction lists, and transaction detail when the transaction can be resolved by `getrawtransaction`.

> The current gap is indexed explorer data. Bitcoin Core RPC by itself does not provide full address history, and `getrawtransaction(txid, true)` only works for arbitrary historical transactions if the provider has `txindex` or equivalent indexing enabled.

Current Tatum fallback endpoints

Endpoint	Why BitScan Uses It
`GET /v3/bitcoin/transaction/{txid}`	Transaction lookup when BitRPC cannot resolve a tx without `block_hash`.
`GET /v3/bitcoin/address/balance/{address}`	Address balance, incoming, outgoing, and pending balance.
`GET /v3/bitcoin/transaction/address/{address}`	Address transaction history with pagination.

What BitRPC Needs To Provide

Option A: Full transaction index support

Support `getrawtransaction(txid, true)` for historical transactions without requiring `block_hash`. This fixes direct transaction searches and transaction pages opened from search.

Option B: Address indexing support

Expose indexed address data that can answer:

- Address confirmed balance
- Address pending balance
- Total received and total sent
- Address transaction history
- Address UTXOs
- Pagination for address transaction history
- Pending or unconfirmed address transactions

Option C: Electrs-compatible API

If BitRPC can expose an Electrs-compatible HTTP API, BitScan can use it for address and transaction indexing.

```
GET /address/{address}
GET /address/{address}/txs
GET /address/{address}/utxo
GET /tx/{txid}
GET /tx/{txid}/status
```

Request To BitRPC Team

BitScan already works well with BitRPC for standard Bitcoin Core methods:

- `getblockchaininfo`
- `getmempoolinfo`
- `getblockhash`
- `getblock`
- `getrawtransaction`

Please confirm whether BitRPC supports:

1. `getrawtransaction(txid, true)` for any historical transaction without requiring `block_hash`.
2. Address balance lookup.
3. Address transaction history lookup.
4. Address UTXO lookup.
5. Pagination for address transaction history.
6. Pending or unconfirmed address transactions.

If these are supported, BitScan can remove Tatum and use BitRPC as the single data source.